



温州肯恩大学

WENZHOU-KEAN UNIVERSITY

MGS 3001 WHS01
Python Programming for Business

Debugging & Error Handling

Dawei Chen

Week 5-1, 24 March 2026

Recap

```
class Person:
    species = "Human"
    def __init__(self, name1, sex1="Male" , profession1):
        self.name = name1
        self.sex = sex1
        self.profession = profession1

    def work(self, company="the company"):
        print(self.name, "is working as a", self.profession, "in", company)

Alice = Person("Alice", "Female", profession="Software Engineer")
Alice.work("Google")
```

1. What are the errors in the above program?

- Parameters **with** default values must **follow** parameters **without** defaults (function definition)
- Keyword arguments** must come **after** **positional arguments** (function call)

Any other error?

2. Why use default parameter values?

- Provide flexibility in function calls
- Allow optional inputs with sensible defaults

A function with
20+ parameters

Recap

3. Can we print the object Alice?

- Output: `<__main__.Person object at 0x...>`
- Meaning: default representation showing object type and memory address

4. Why not a readable output like `[1, 2, 3]`?

- Built-in types (e.g., list) define readable string formats
- Custom objects do not unless explicitly defined (e.g., `__str__` or `__repr__`)

5. How is data stored in objects?

Built-in Objects (e.g., list, dict, tuple)

- Data is stored in predefined internal structures
- Access via indexing or keys, not attributes
- Structure and behavior are fixed by Python
 - `list`: ordered sequence of elements → `[1, 2, 3]`
 - `dict`: key-value pairs → `{"a": 1, "b": 2}`

Custom Objects (user-defined classes)

- Data is stored as attributes (variables bound to the object)
- Accessed via dot notation: `.name`, `.sex`
- Defined explicitly by the programmer in a class

Recap

```
class Product:
    category = "General"

    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def display_info(self):
        print(self.name, self.price, self.stock)
```

1) What is the difference between class attribute and object attributes in this code?

2) If we create two objects:

```
p1 = Product("Laptop", 8000, 10)
p2 = Product("Phone", 5000, 20)
```

- Which attributes are shared?
- Which are different? Why?

3) What will happen if we run:

```
p1.category = "Electronics"
print(p1.category, p2.category)
```

Explain the result.

4) What will happen if we run:

```
Product.category = "Electronics"
print(p1.category, p2.category)
```

Explain the difference from Q3.

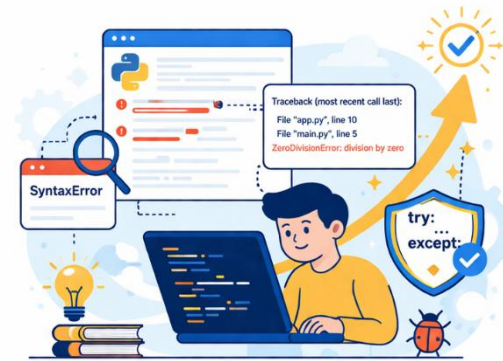
5) Predict the output

```
p1.display_info()
```

Learning Objectives

By the end of this session, students will be able to:

- Understand common Python errors (syntax, runtime, logical)
- Interpret error messages (tracebacks) to locate issues
- Apply systematic debugging strategies
- Use `try/except` to manage exceptions
- Write more robust Python programs



Generated by ChatGPT Auto

Why Debugging Matters

How long have you spent debugging a single bug?



It takes up most of your time

Studies of professional developers show debugging accounts for ~50% of total programming time — more than writing new code.



Errors are expected

Every working program started with bugs. Writing code without errors is not the goal — finding and fixing them efficiently is.



Understanding beats guessing

Randomly deleting lines is not debugging. The goal is to understand *why* an error occurred — not just to make the red message disappear.

Good programmers are not those who don't make errors — but those who can debug efficiently.

Types of Errors in Python



Generated by ChatGPT Auto

Syntax Error

- Python cannot parse the code because it violates the language's grammar rules
- Caught **before** execution, code never runs

Common Causes

- Missing colon after `if`, `for`, `def`
- Wrong or missing indentation
- Unclosed bracket or parenthesis
- Misspelled keyword (e.g., `while`, `dfe`)
- Illegal characters (e.g., Chinese input)

- Easy to fix
- IDE auto-detection
- Red underline / error makers / suggested fixed

Types of Errors in Python

Runtime Error (Exception)

- Code is syntactically correct, but an error occurs during execution
- Raised **during** execution; code starts, then crashes

Common Types

- `ZeroDivisionError` — division by zero
- `TypeError` — incompatible data types
- `NameError` — variable not defined
- `IndexError` / `KeyError` — invalid access
- `ValueError` — right type, wrong value
- `FileNotFoundError` — file does not exist



Generated by ChatGPT Auto

- Easy to fix
- Use traceback to locate the error
- apply try / except for handling

Built-in Exception

Types of Errors in Python

Logical Error

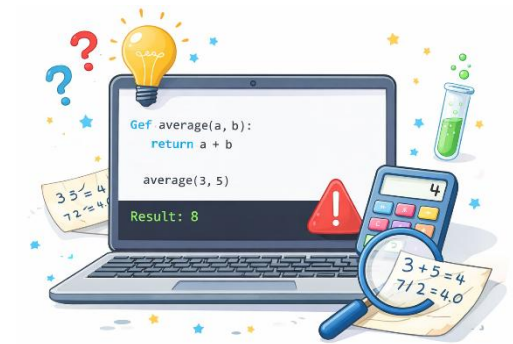
- **No error message** is raised; code runs but result is wrong / unexpected
- Python cannot detect, only you can, through testing and reasoning

Common Causes

- Using `=` instead of `==`
- Off-by-one in a loop (`<` vs. `≤`)
- Incorrect formula or logic
- Wrong condition or operator
- Operations in the wrong order

```
total = 420
count = 10
avg = total / count + 1
avg # no error, but wrong!
# should be: total / (count + 1)
```

- Difficult to fix
- Check logic step-by-step
- Use print/logging
- Test with simple cases



Generated by ChatGPT Auto

Tracebacks

- An error report showing **where** and **why** a program failed
- Key components
 - File name & line number → where the error occurred
 - Code snippet → the problematic line
 - Error type & message → what went wrong
- How to read a traceback
 - Read from bottom → top
 - Start with the final error message
 - Then trace back to locate the source

Don't ignore error messages, they are the best debugging tool.

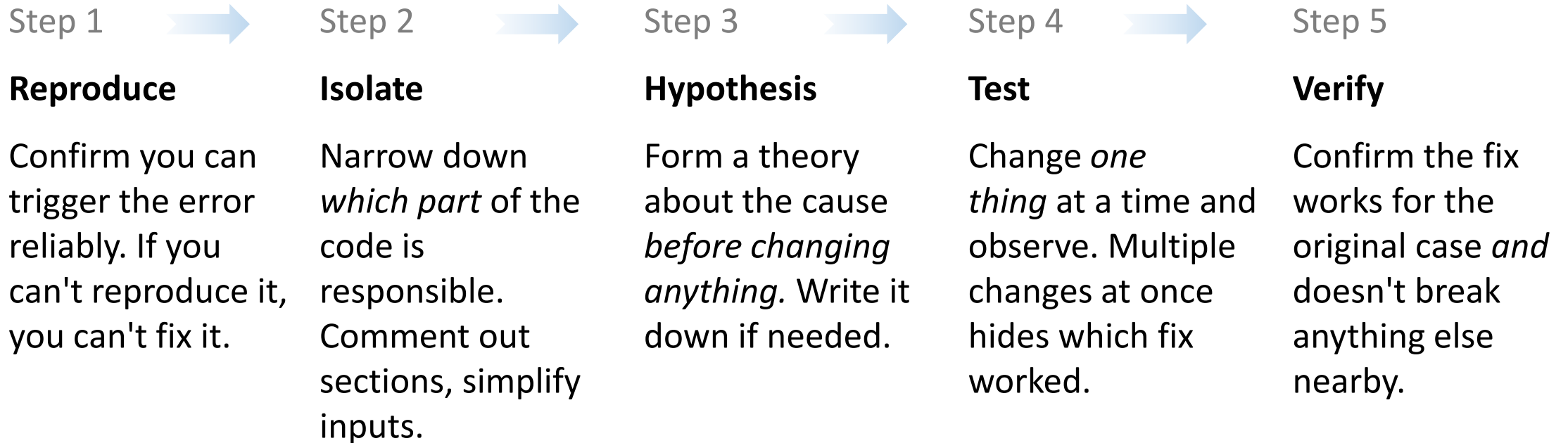
```
• numbers = [10, 20, 30]
  print(numbers[3])
✖ 0.0s

-----
IndexError                                Traceback (most recent call last)
Cell In[19], line 2
      1 numbers = [10, 20, 30]
----> 2 print(numbers[3])

IndexError: list index out of range
```

Debugging Strategies

A systematic approach beats random guessing every time — here's the process and the tools



Practical Debugging Techniques

`print()` debugging

- Add strategic print statements
- Inspect values: `print(x)`
- Check type/structure: `type(x), len(x)`
- Add context: `print("value of x:", x)`

```
# print value AND type
print(type(x), x)

# label prints clearly
print("before loop:", items)
# ... loop body ...
print("after loop: ", items)
```

Assert statements

- Embed assumptions in code
- Fail fast if condition is violated
- Prevent silent wrong output
- Provide clear error message

```
# check assumptions early
assert isinstance(scores, list), \
    "scores must be a list"
assert len(scores) > 0, \
    "list cannot be empty"

result = sum(scores) / len(scores)
```

IDE debugger

- Set breakpoints
- Step through code line by line
- Inspect variables in real time
- More powerful than `print()`

```
# VS Code / PyCharm:
# click the line number gutter
# → red dot appears
# → run in debug mode
# Step over  F10
# Step into  F11
# Inspect   hover var
```

Error Handling: try / except

Purpose

- Handle runtime errors (exceptions)
- Prevent program from crashing
- Provide meaningful feedback

```
x = int("abc")
```

```
try:  
    x = int("abc")  
except ValueError:  
    print("Cannot convert to integer")
```

```
try:  
    x = 10 / 0  
except ValueError:  
    print("Value error happened")
```

Basic Structure

- try → risky code that may fail
- except → handle the error

```
try:  
    x = 10 / 0  
except ValueError:  
    print("Value error")  
except ZeroDivisionError:  
    print("Cannot divide by zero")
```

```
try:  
    x = int("5")  
except ValueError:  
    print("Conversion failed")  
else:  
    print("Conversion successful:", x)  
finally:  
    print("Program finished")
```

In-Class Exercise

- Download `error_handling.ipynb` in Canvas
- Open the notebook in VS Code (Jupyter)
- Follow the instructions in the notebook



Generated by ChatGPT Auto

Summary

Types of Errors

- Syntax → code cannot run
- Runtime → program crashes
- Logical → wrong output

Debugging Process

- Reproduce → Isolate → Hypothesis → Test → Verify

Key Techniques

- `print()` → inspect values
- `assert` → check assumptions
- Debugger → step through code

Error Handling (`try/except`)

- Catch specific exceptions
- Handle errors gracefully
- Provide clear messages

Q&A

CBPM B214

dchen@kean.edu

Tue & Thu 11:15 – 12:30 | Wed 14:00 – 16:30